What is type parameter <T> in Java :

* It is by to make java application **independent of data type,** Java introduced Type parameter (Generics) from JDK 1.5 onwards.

* It is represented by **<T>** which can accept any Wrapper class OR User-defined class but not the **primitive data type**.

Example :

```
public class Box<T>  //Here T represents the Type parameter
{
}
}
```

```
//Program :
//--------
```

Is this Box we can accept any type of data.

```
package com.ravi.type_parameter;

class Box<T> //Type Parameter (Generic)
{
    private T data;

    public Box(T data)  //Student data = new Student(1,"A");
    {
        super();
        this.data = data;
    }

    public T getData()
    {
        return this.data;
    }
}

public class TypeParameterDemo
{
    public static void main(String[] args)
    {

        BoxInteger intType = new BoxInteger(12);
        System.out.println("Integer Type is :"+intType.getData());

        BoxDouble doubleType = new BoxDouble(2500);
        System.out.println("Double Type is :"+doubleType.getData());

        BoxCharacter charType = new BoxCharacter('a');
        System.out.println("Character Type is :"+charType.getData());

        BoxStudent stuType = new BoxStudent(new Student(1, "Ra[7]");
        System.out.println("Student Type is :"+stuType.getData());

    }
}

class Student
{
    private int id;
    private String name;

    public Student(int id, String name)
    {
        super();
        this.id = id;
        this.name = name;
    }

    @Override
    public String toString();
    {
        return "Student [id= " + id + ", name=" + name + "]";
    }
}
```

Working with functional interface to write lambda :

```
package com.ravi.functional_interface;

//Verify whether a number is even or odd

@FunctionalInterface
interface PredicateOne
{
    boolean verify(int num);
}

public class FunctionalInterfaceDemo {

    public static void main(String[] args)
    {
        PredicateOne p = - num -> num %2==0;
        int num = Integer.parseInt(JOptionPane.showInputDialog("Enter a number :"));
        JO.println("Is :"+num+" Even number ?"+p.verify(num));

    }

}
```

Working with predefined functional interfaces :

* Java has provided the following **predefined functional interfaces** available in java.util.function sub package.

* These interfaces will provide concise code writing in our day-to-day programming.

1) Predicate<T>
2) Consumer<T>
3) BiConsumer<T,U>
4) Supplier<T>
5) Function<T,R>
6) BiFunction<T,U>
7) UnaryOperator<T>
8) BinaryOperator<T>

Note : The have total 43+ functional interfaces are available.

Predicate<T> Functional interface :

* It is a predefined functional interface available in java.util.function sub package.

```
@FunctionalInterface
public interface Predicate<T>
{
    boolean test( );
}
```

* It accepts an abstract method test() which is accepting T type of parameter and return type is boolean.

* IT IS MAINLY USED TO VERIFY 'ONE ARGUMENT BOOLEAN EXPRESSION'

```
//Program :
package com.ravi.predicate;

import java.util.function.Predicate;

//Verify another a person is eligible for vote or not

public class PredicateDemo2
{
    public static void main(String[] args)
    {
        Predicate<Integer> p1 = age -> age >= 18;

        int age = Integer.parseInt(JO.showInput("Enter your age :"));

        boolean isEligible = p1.test(age);

        if(isEligible)
        {
            JO.println("Eligible for Vote");
        }
        else
        {
            JO.println("Not Eligible for Vote");
        }
    }

}

package com.ravi.predicate;

import java.util.function.Predicate;

//Verify person is user starts with character s or not

public class PredicateDemo3
{
    public static void main(String[] args)
    {
        Predicate<String> p1 = name -> name.startWith("S");
        String name = JO.readS("Enter your Name :");

        boolean isUser = p1.test(name);

        if(isUser)
        {
            JO.println("starts with S");
        }
        else
        {
            JO.println("does not start with S");
        }
    }

}

package com.ravi.predicate;

import java.util.function.Predicate;

//Verify whether Student is pass OR fail

class Student
{
    private int id;
    private String name;
    private double marks;

    public Student(int id, String name, double marks)
    {
        super();
        this.id = id;
        this.name = name;
        this.marks = marks;
    }


    public int getId( )
    {
        return id;
    }


    public void setId(int id) {
        this.id = id;
    }


    public String getName() {
        return name;
    }


    public void setName(String name) {
        this.name = name;
    }


    public double getMarks() {
        return marks;
    }


    public void setMarks(double marks) {
        this.marks = marks;
    }


    @Override
    public String toString()
    {
        return "Student [id=" + id + ", name=" + name + ", marks=" + marks + "]";
    }
}

public class PredicateDemo4 {

    public static void main(String[] args)
    {
        Predicate<Student> p1 = student -> student.getMarks()>=75;

        int id = Integer.parseInt(JO.readS("Enter Student ID :"));
        String name = JO.readS("Enter Student name :");
        double marks = Double.parseDouble(JO.readS("Enter Student marks :"));

        Student stu = new Student(id, name, marks);

        boolean isPass = p1.test(stu);

        if(isPass)
        {
            JO.println(stu.getName()+" is passed");
        }
        else
        {
            JO.println(stu.getName()+" is failed");
        }
    }

}
```

Define program, We are re-writing for multiple students :

```
package com.ravi.predicate;

import java.util.function.Predicate;

//Verify another Student is pass OR not

class Student
{
    private int id;
    private String name;
    private double marks;

    public Student(int id, String name, double marks)
    {
        super();
        this.id = id;
        this.name = name;
        this.marks = marks;
    }

    public int getId( )
    {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getMarks() {
        return marks;
    }

    public void setMarks(double marks) {
        this.marks = marks;
    }

    @Override
    public String toString()
    {
        return "Student [id=" + id + ", name=" + name + ", marks=" + marks + "]";
    }
}

public class PredicateDemo5 {

    public static void main(String[] args)
    {
        Predicate<Student> p1 = student -> student.getMarks()>=75;

        Student [] students = new Student[5];
        students[0] = new Student(1, "Ra[7]", 90);
        students[1] = new Student(2, "Ram", 75);
        students[2] = new Student(3, "Ram", 72);
        students[3] = new Student(4, "Ram", 65);
        students[4] = new Student(5, "Ram", 90);

        for(Student student : students)
        {
            if(p1.test(student))
            {
                JO.println(student.getName()+" is passed");
            }
            else
            {
                JO.println(student.getName()+" is not Passed");
            }
        }

    }

}
```

Assignment :

Create a class called Car

```
                    name
                    model
                    price
                    mileage
```

Write a PredicateOne, we can get out the car on the following criteria :

a) Car price should be **less than 5 Lakh** and **mileage must be >=20**